

COMPSCI 752

XPath

Big data management

Instructors: Sebastian Link and Ninh Pham

Outline

- 1 Introduction
- 2 Path Expressions
- 3 Operators and Functions
- 4 XPath examples
- 5 XPath 2.0, 3.0, and 3.1
- 6 Reference Information
- 7 Exercise

XPath

- A language of expressions for use in another host language (e.g., XSLT, XQuery)
- Describes paths in an XML tree, and the retrieval of nodes that match these paths
- Can also be used for performing some (limited) operations on XML data

有限操作数据.

XPath

- A language of expressions for use in another host language (e.g., XSLT, XQuery)
- Describes paths in an XML tree, and the retrieval of nodes that match these paths
- Can also be used for performing some (limited) operations on XML data

Example

- $2 * 3$ is an XPath literal expression
- $// *$ [@msg="Hello world"] is an XPath path expression, retrieving all elements with a msg attribute set to "Hello world"

字面量表达式 → 直接计算数值、布尔、字符串，不含XML结构

选取所有匹配元素 所有元素

查找XML文档中所有msg属性为Hello world的元素

@用于选择XML元素的属性

XPath

- A **language of expressions** for use in another host language (e.g., XSLT, XQuery)
- Describes **paths** in an XML tree, and the retrieval of nodes that match these paths
- Can also be used for performing some (limited) operations on XML data

Example

- $2 * 3$ is an XPath **literal expression**
- `//*[@msg="Hello world"]` is an XPath **path expression**, retrieving all elements with a `msg` attribute set to "Hello world"

Content of this presentation

Mostly XPath 1.0: a W3C recommendation published in 1999, widely used

Also a *basic* introduction to XPath 2.0, published in 2007

XPath Data Model

XPath expressions operate over **XML trees**, which consist of the following **node types**:

- **Document**: the root node of the XML document;
- **Element**: element nodes;
- **Attribute**: attribute nodes, represented as children of an **Element** node;
- **Text**: text nodes, i.e., leaves of the XML tree.

XPath Data Model

XPath expressions operate over **XML trees**, which consist of the following **node types**:

- **Document**: the **root node** of the XML document;
- **Element**: element nodes;
- **Attribute**: attribute nodes, represented as children of an **Element** node; 有名称, 无值
- **Text**: text nodes, i.e., leaves of the XML tree. 无名称, 有值

Remark

- The XPath data model features also **ProcessingInstruction** and **Comment** node types 处理指令节点 注释节点
- Syntactic features specific to serialized representation (e.g., entities, literal section) are ignored by XPath 语法的特征 序列化的

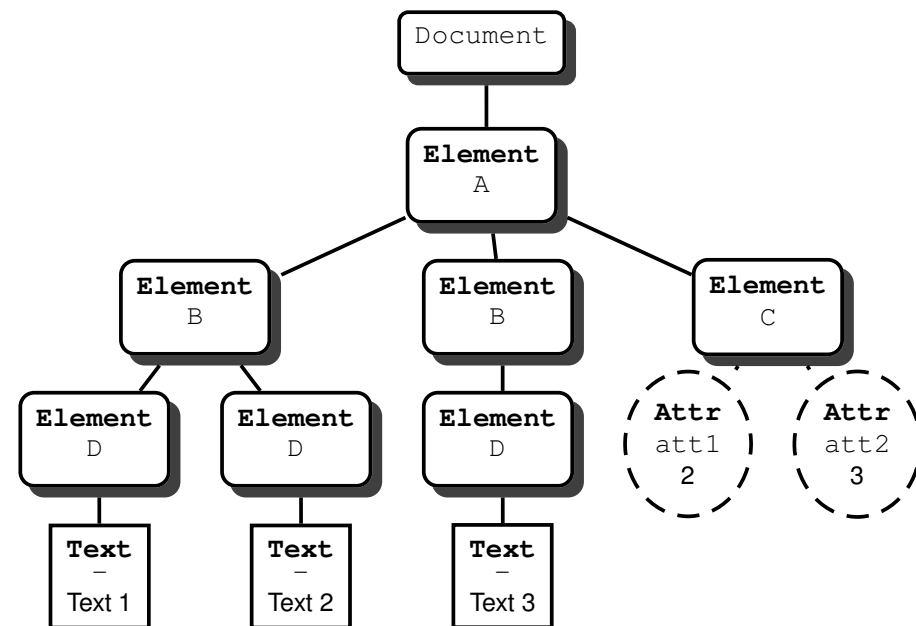
From serialized representation to XML trees

```
<?xml version="1.0"
      encoding="utf-8"?>
<A>
  <B att1='1'>
    <D>Text 1</D>
    <D>Text 2</D>
  </B>
  <B att1='2'>
    <D>Text 3</D>
  </B>
  <C att1="2"
      att2="3"/>
</A>
```

(Some attributes are not shown)

From serialized representation to XML trees

```
<?xml version="1.0"
      encoding="utf-8"?>
<A>
  <B att1='1'>
    <D>Text 1</D>
    <D>Text 2</D>
  </B>
  <B att1='2'>
    <D>Text 3</D>
  </B>
  <C att1="2"
      att2="3"/>
</A>
```



(Some attributes are not shown)

XPath Data Model (cont.)

- The root node of an XML tree is the (unique) Document node
- The root element is the (unique) Element child of the root node.
- A node has a name, or a value, or both
 - ▶ an **Element** node has a name, but no value
 - ▶ a **Text** node has a value (a character string), but no name
 - ▶ an **Attribute** node has both a name and a value
- *Attributes are special:* They are not considered as first-class nodes in an XML tree; They must be addressed specifically when needed

XPath Data Model (cont.)

- The **root node** of an XML tree is the (unique) **Document** node
- The **root element** is the (unique) **Element** child of the root node
- A node has a **name**, or a **value**, or both
 - ▶ an **Element** node has a name, but no value
 - ▶ a **Text** node has a value (a character string), but no name
 - ▶ an **Attribute** node has both a name and a value
- *Attributes are special*: They are not considered as first-class nodes in an XML tree; They must be addressed specifically when needed

Remark

The expression “*textual value of an **Element** N* ” denotes the concatenation of all the **Text** node values which are descendants of N , taken in **document order**

Outline

1 Introduction

2 Path Expressions

- Steps and expressions
- Axes and node tests
- Predicates

3 Operators and Functions

4 XPath examples

5 XPath 2.0, 3.0, and 3.1

6 Reference Information

values which are descendants of N, taken in document order 这句话是什么意思

XPath 2.0,
Reference
Exercise

这句话的意思是：

“元素 N 的文本值”表示所有属于 N 的后代 (descendants) 的文本节点 (Text nodes) 的值按照文档顺序 (document order) 连接 (concatenation) 在一起的结果。

换句话说，如果一个 XML 元素包含多个文本节点 (可能是直接包含的，也可能是嵌套的子元素中的文本)，那么这个元素的“文本值”就是所有这些文本节点的内容按照 XML 文档中的顺序拼接起来的字符串。

示例：

假设有如下 XML 结构：

```
xml
<A>
  <B>Text 1</B>
  <B>
    <C>Text 2</C>
    <C>Text 3</C>
  </B>
</A>
```

对于元素 <A>，它的“文本值”就是：

```
arduino
"Text 1Text 2Text 3"
```

因为它的所有文本节点是 Text 1、Text 2 和 Text 3，并且它们按文档顺序依次排列。

这种定义对于 XPath 这样的查询语言非常重要，因为 XPath 查询 string(A) 或

XPath Context 上下文

A step is evaluated in a specific **context** $[\langle N_1, N_2, \dots, N_n \rangle, N_c]$ which consists of:

- A **context list** $\langle N_1, N_2, \dots, N_n \rangle$ of nodes from the XML tree
- A **context node** N_c belonging to the context list

正在处理的节点

XPath Context

A step is evaluated in a specific **context** $[< N_1, N_2, \dots, N_n >, N_c]$ which consists of:

- A **context list** $< N_1, N_2, \dots, N_n >$ of nodes from the XML tree
- A **context node** N_c belonging to the context list

Information on the context

- The **context length** n is a positive integer indicating the **size** of a contextual list of nodes; it is output of the function `last()` 返回文本长度, $<n_1, n_2, n_3>$ 返回 3
- The **context node position** $c \in [1, n]$ is a positive integer indicating the **position** of the context node in the context list of nodes; it is output of the function `position()` 返回正在处理节点的位置

XPath steps

The basic component of XPath expressions are **steps**, and are of the form:

$\text{axis} :: \text{node-test} [P_1] [P_2] \dots [P_r]$

- **axis** is an **axis name** indicating what the direction of the step in the XML tree is (**child** is the default)
- **node-test** is a **node test**, indicating the kind of nodes to select
- **P_i** is a **predicate**, that is, any XPath expression evaluated to **boolean**, indicating an additional condition; there may be no predicates at all

XPath steps

The basic component of XPath expressions are **steps**, and are of the form:

$$\text{axis} :: \text{node-test} [P_1] [P_2] \dots [P_n]$$

- **axis** is an **axis name** indicating what the direction of the step in the XML tree is (**child** is the default)
- **node-test** is a **node test**, indicating the kind of nodes to select
- P_i is a **predicate**, that is, any XPath expression evaluated to `boolean`, indicating an additional condition; there may be no predicates at all

解释

Interpretation of a step

A step is evaluated with respect to a **context**, and returns a **list of nodes**

步骤根据上下文计算的，返回一个节点列表。

XPath steps

The basic component of XPath expressions are **steps**, and are of the form:

$$\text{axis} :: \text{node-test} [P_1] [P_2] \dots [P_n]$$

- **axis** is an **axis name** indicating what the direction of the step in the XML tree is (**child** is the default)
- **node-test** is a **node test**, indicating the kind of nodes to select
- P_i is a **predicate**, that is, any XPath expression evaluated to `boolean`, indicating an additional condition; there may be no predicates at all

Interpretation of a step

A step is evaluated with respect to a **context**, and returns a **list of nodes**

Example

`descendant::C[@att1='1']` is a step which returns all **Element** nodes named `C` that are descendants of the context node, and have an **Attribute** node `att1` with value `1`

Path Expressions

A path expression is of the form: $[/]\text{step}_1 / \text{step}_2 / \dots / \text{step}_n$

- A path that begins with $/$ is an absolute path expression;
- A path that does not begin with $/$ is a relative path expression

从根节点查询 绝对路径
上下文节点查询
相对路径

Path Expressions

A path expression is of the form: $[/]\text{step}_1 / \text{step}_2 / \dots / \text{step}_n$

- A path that begins with `/` is an **absolute** path expression;
- A path that does not begin with `/` is a **relative** path expression

Example

- `/A/B` is an **absolute** path expression 指示 denoting the **Element** nodes with name `B`, which are children of the root named `A`
- `./B/descendant::text()` is a **relative** path expression which denotes all the **Text** node descendants of an **Element** `B`, itself child of the context node
- `/A/B/@att1[.>2]` 当前节点 denotes all the **Attribute** nodes `@att1` (of a `B` node child of the `A` root element) whose value is greater than 2

Path Expressions

A path expression is of the form: $[/]\text{step}_1 / \text{step}_2 / \dots / \text{step}_n$

- A path that begins with `/` is an **absolute** path expression;
- A path that does not begin with `/` is a **relative** path expression

Example

- `/A/B` is an **absolute** path expression denoting the **Element** nodes with name `B`, which are children of the root named `A`
- `./B/descendant::text()` is a **relative** path expression which denotes all the **Text** node descendants of an **Element** `B`, itself child of the context node
- `/A/B/@att1[. > 2]` denotes all the **Attribute** nodes `@att1` (of a `B` node child of the `A` root element) whose value is greater than 2

`.` is a special step, which refers to the context node. e.g. `./toto` means the same thing as `toto`

Evaluation of Path Expressions

Each step step_i is interpreted with respect to a **context**; its result is a **node list**

A step step_i is evaluated with respect to the context of step_{i-1} . More precisely:

Evaluation of Path Expressions

Each step step_i is interpreted with respect to a **context**; its result is a **node list**

A step step_i is evaluated with respect to the context of step_{i-1} . More precisely:

- For $i = 1$ (first step) if the path is **absolute**, the context is a singleton, the root of the XML tree; else (**relative** paths) the context is defined by the environment
- For $i > 1$ if $\mathcal{N} = \langle N_1, N_2, \dots, N_n \rangle$ is the result of step step_{i-1} , step_i is successively evaluated with respect to the context $[\mathcal{N}, N_j]$, for each $j \in [1, n]$

Evaluation of Path Expressions

解释

Each step step_i is interpreted with respect to a **context**; its result is a **node list**

A step step_i is evaluated with respect to the context of step_{i-1} . More precisely:

Step i 的上下文由 Step i-1 (上一步) 决定

例子

- For $i = 1$ (first step) if the path is **absolute**, the context is a singleton, the **root** of the XML tree; else (**relative** paths) the context is defined by the environment
- For $i > 1$ if $\mathcal{N} = \langle N_1, N_2, \dots, N_n \rangle$ is the result of step step_{i-1} , step_i is successively evaluated with respect to the context $[\mathcal{N}, N_j]$, for each $j \in [1, n]$

The result of the path expression is the node set obtained after evaluating the last step

3. 评估过程:

- 对于每个 N_j , XPath引擎会将其作为当前上下文节点, 执行当前步骤的操作 (如选择子节点、属性等)。
- 所有节点的操作结果会被合并, 形成下一步的上下文列表。

示例说明

假设路径表达式是 `/a/b/c`, XML结构如下:

```
xml                                                                    复制
<a>
  <b>
    <c>1</c>
  </b>
  <b>
    <c>2</c>
  </b>
</a>
<a>
  <b>
    <c>3</c>
  </b>
</a>
```

▶ 运行 HTML

1. 第一步:

- 路径 `/a` 的结果是节点列表 $\mathcal{N} = \langle a1, a2 \rangle$ 。

2. 第二步:

- 对于每个 `<a>` 节点 (即 N_j):
 - 以 $[\langle a1, a2 \rangle, a1]$ 为上下文, 查找 `a1` 的子节点 `<b>`, 结果是 `<b1, b2>`。
 - 以 $[\langle a1, a2 \rangle, a2]$ 为上下文, 查找 `a2` 的子节点 `<b>`, 结果是 `<b3>`。
- 合并结果: $\mathcal{N} = \langle b1, b2, b3 \rangle$ 。

3. 第三步:

- 对于每个 `<b>` 节点 (即 N_j):
 - 以 $[\langle b1, b2, b3 \rangle, b1]$ 为上下文, 查找 `b1` 的子节点 `<c>`, 结果是 `<c1>`。
 - 以 $[\langle b1, b2, b3 \rangle, b2]$ 为上下文, 查找 `b2` 的子节点 `<c>`, 结果是 `<c2>`。
 - 以 $[\langle b1, b2, b3 \rangle, b3]$ 为上下文, 查找 `b3` 的子节点 `<c>`, 结果是 `<c3>`。
- 合并结果: $\mathcal{N} = \langle c1, c2, c3 \rangle$ 。

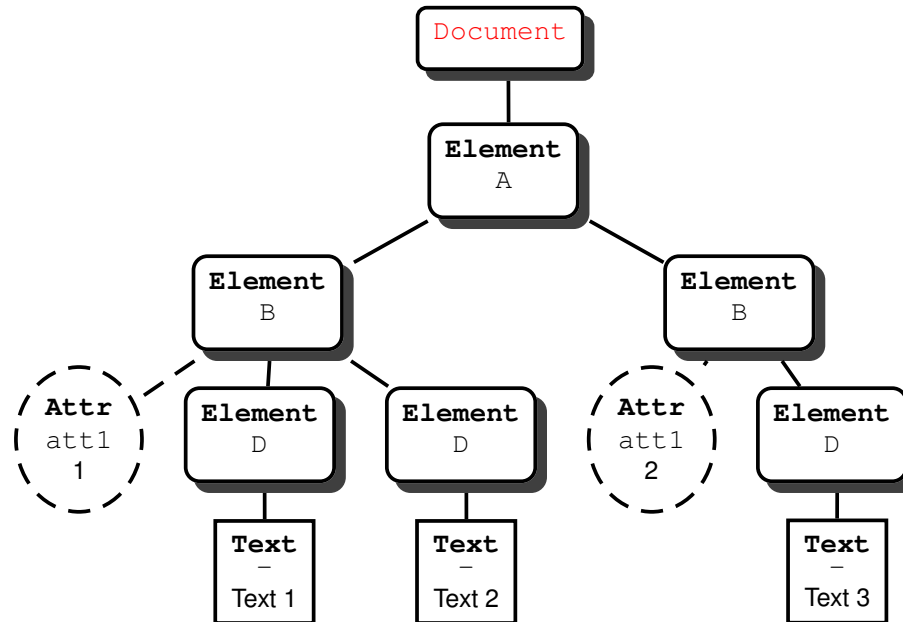
4. 最终结果:

- 路径表达式的结果是所有 `<c>` 节点的集合: `<c1, c2, c3>`。

Evaluation of `/A/B/@att1`

The path expression is absolute: the context consists of the root node of the tree

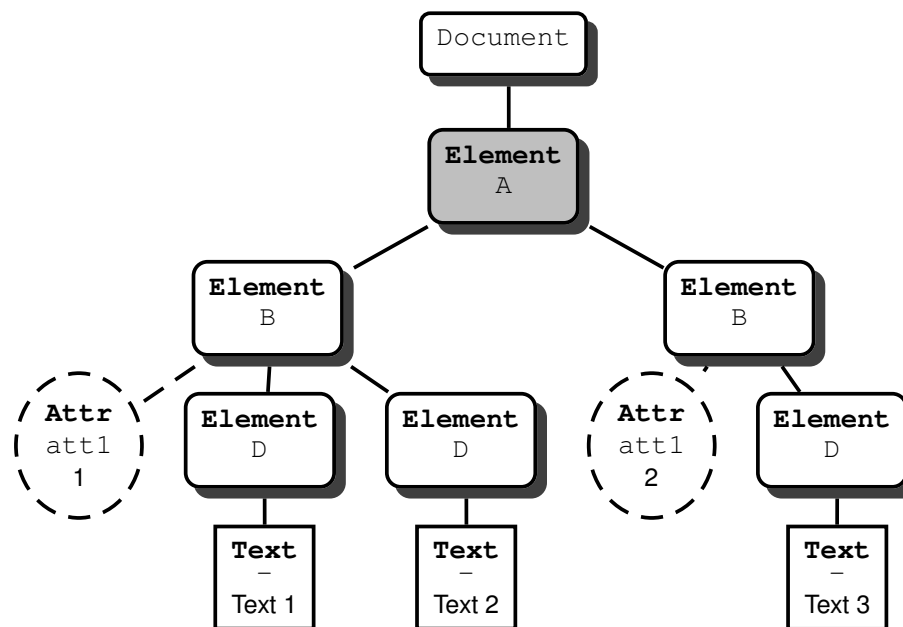
The first step, `A`, is evaluated with respect to this context.



Evaluation of `/A/B/@att1`

The result is **A**, the root element

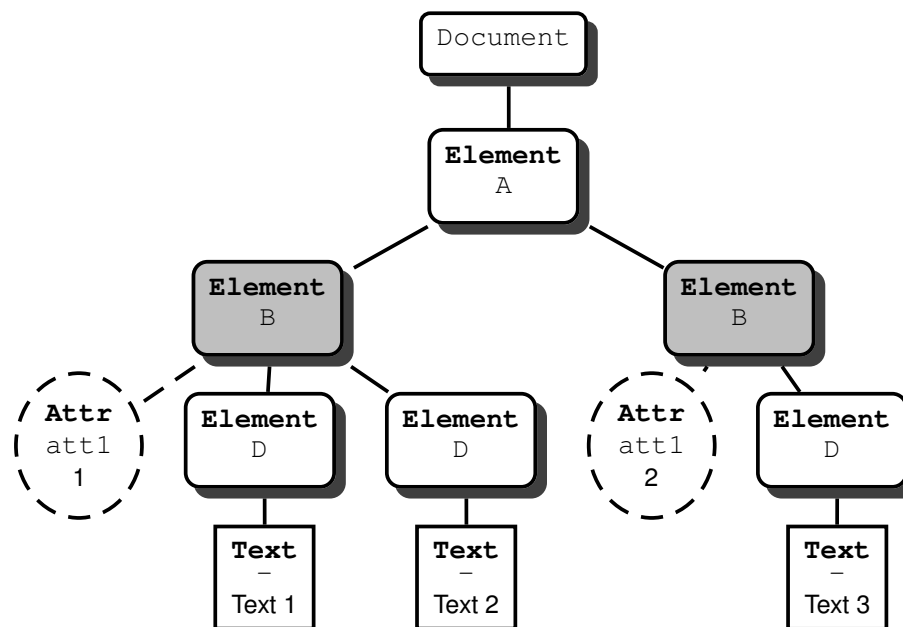
A is the context for the evaluation of the second step, B.



Evaluation of `/A/B/@att1`

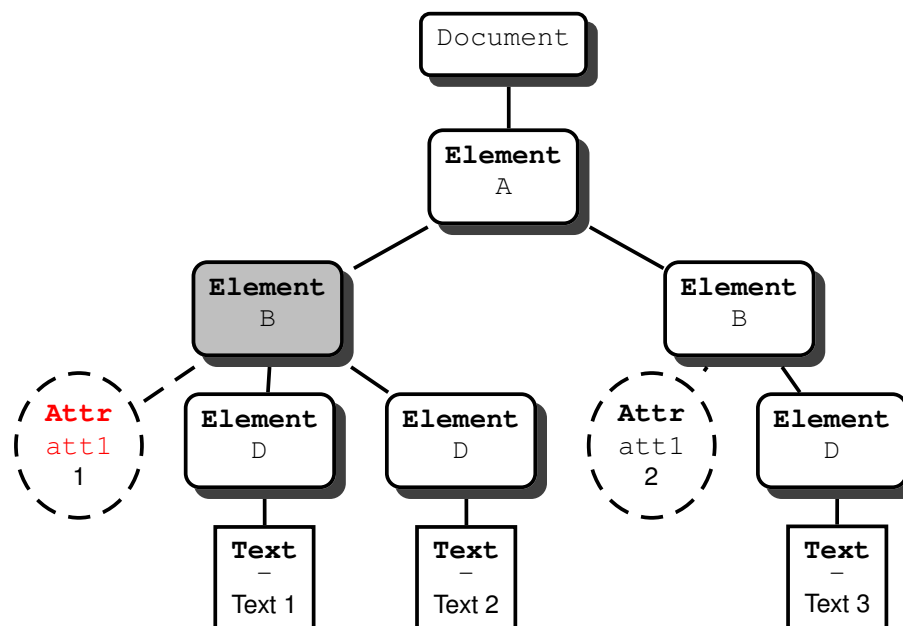
The result is a node list with two nodes **B[1], B[2]**

`@att1` is first evaluated with
the context node **B[1]**.



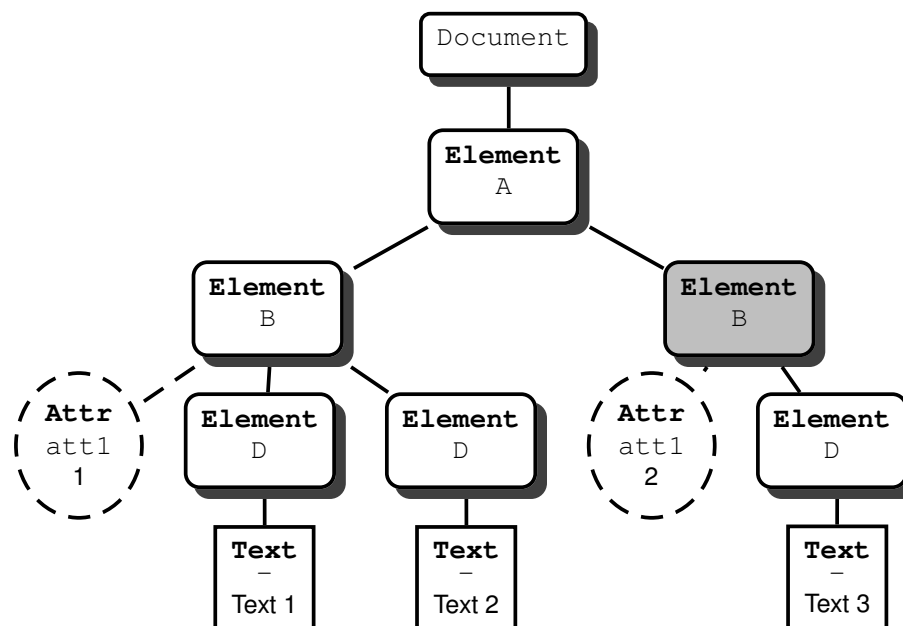
Evaluation of `/A/B/@att1`

The result is the attribute node of **B[1]**



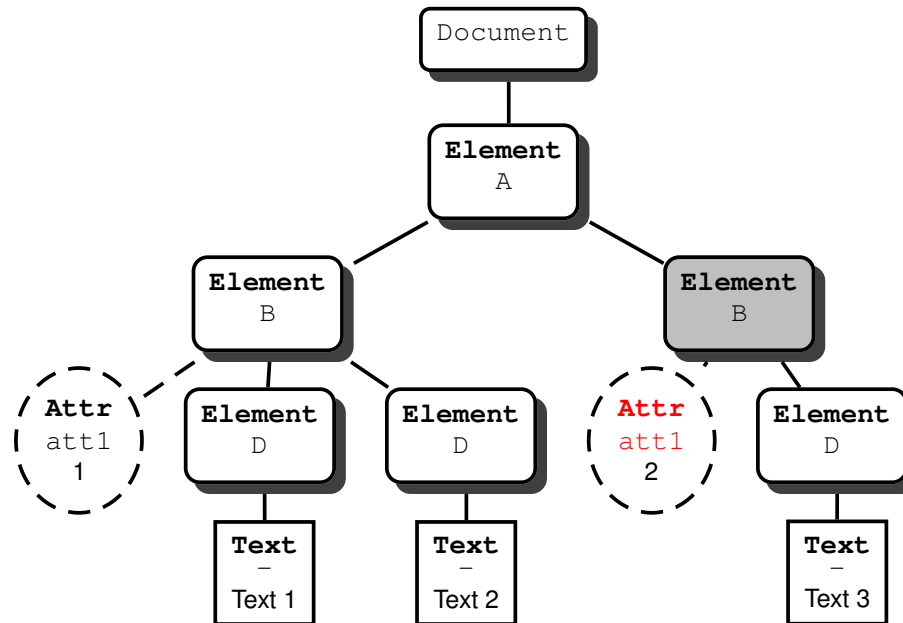
Evaluation of `/A/B/@att1`

`@att1` is also evaluated with
the context node **B[2]**.



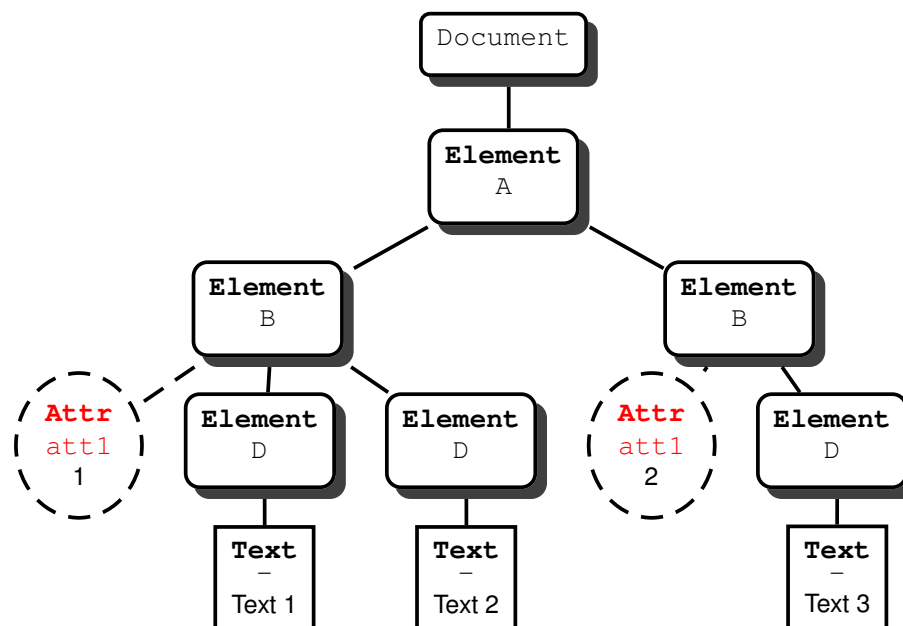
Evaluation of `/A/B/@att1`

The result is the attribute node of **B[2]**



Evaluation of `/A/B/@att1`

Final result: the node set union of all the results of the last step, `@att1`



Axes

An axis: A set of nodes determined from the context node, **and** an ordering of the sequence

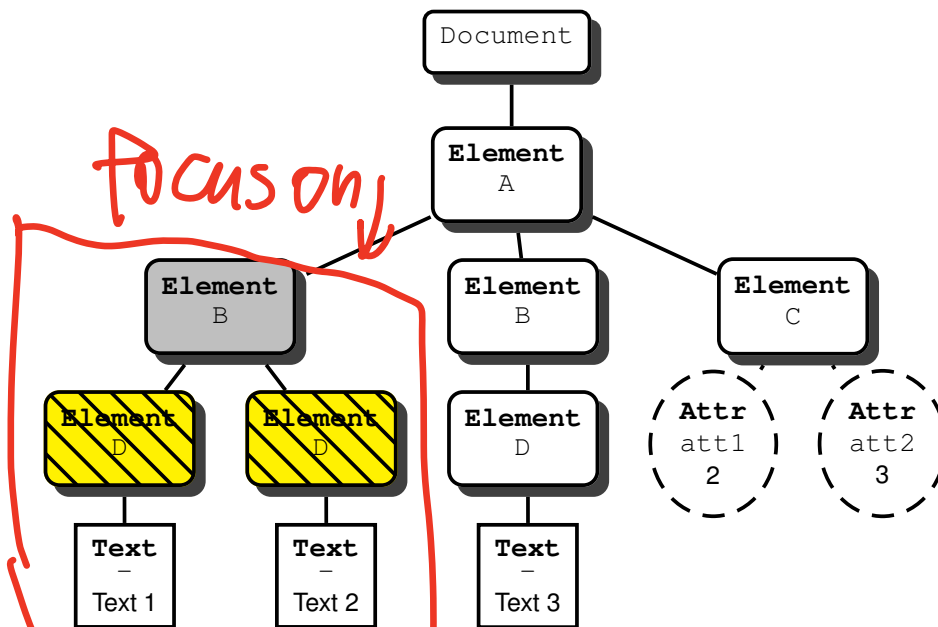
- `child` (**default axis**)
- `parent` Parent node
- `attribute` Attribute nodes
- `descendant` Descendants, excluding the node itself
- `descendant-or-self` Descendants, including the node itself
- `ancestor` Ancestors, excluding the node itself
- `ancestor-or-self` Ancestors, including the node itself
- `following` Following nodes in **document order**
- `following-sibling` Following siblings in **document order**
- `preceding` Preceding nodes in **document order**
- `preceding-sibling` Preceding siblings in **document order**
- `self` The context node itself

Examples of axis interpretation

Result of `child::D` (equivalent to `D`)

Child axis:

- Denotes the **Element** or **Text** children of the context node
- **Important:** An **Attribute** node has a parent (the element on which it is located), but an attribute node is *not* one of the children of its parent



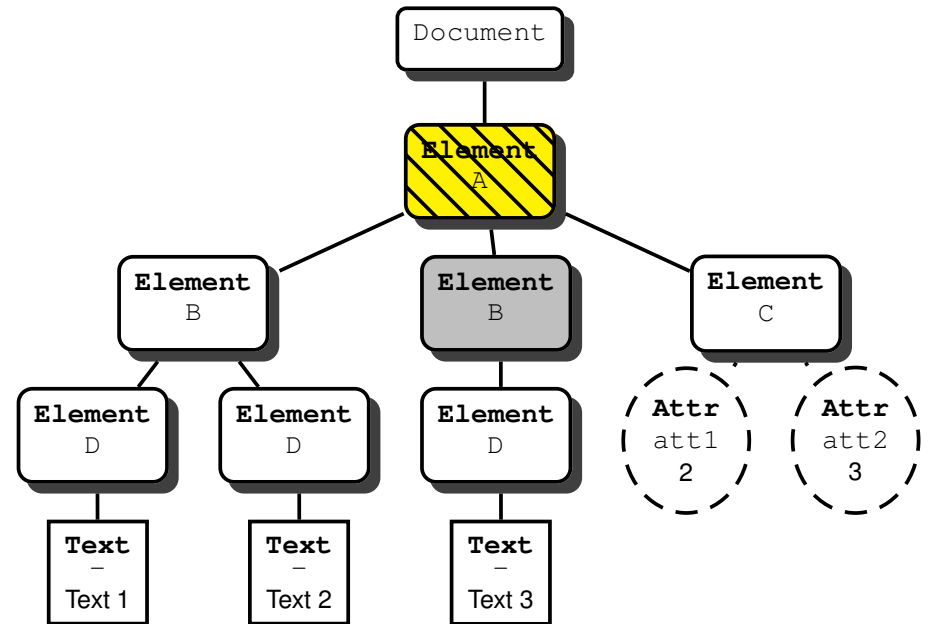
属性节点不被
视为子节点, 有父节点

Examples of axis interpretation

Parent axis:

- Denotes the parent of the context node
- Node test is either an element name, or `*` which matches all names, or `node()` which matches all node types
- Always an **Element** or **Document** node, or an empty node set (if the parent does not match the node test or does not satisfy a predicate)
- `..` is an abbreviation for `parent::node()`: the parent of the context node, whatever its type

Result of `parent::node()` (or short: `..`)

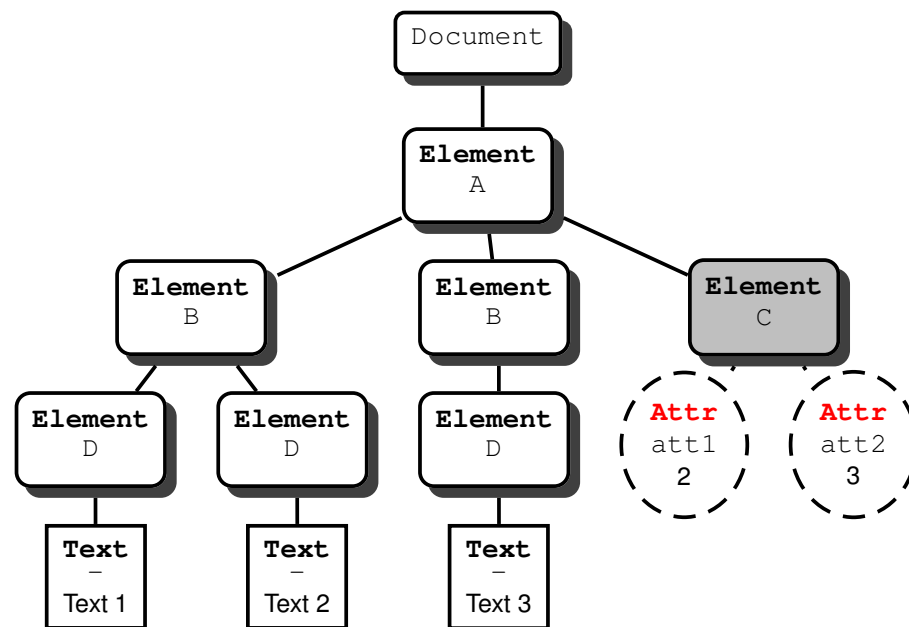


Examples of axis interpretation

Attribute axis:

- Denotes the attributes of the context node
- The node test is either the attribute name, or `*` which matches all the names

Result of **attribute::*** (short: **@***)



Examples of axis interpretation

测试

Descendant axis:

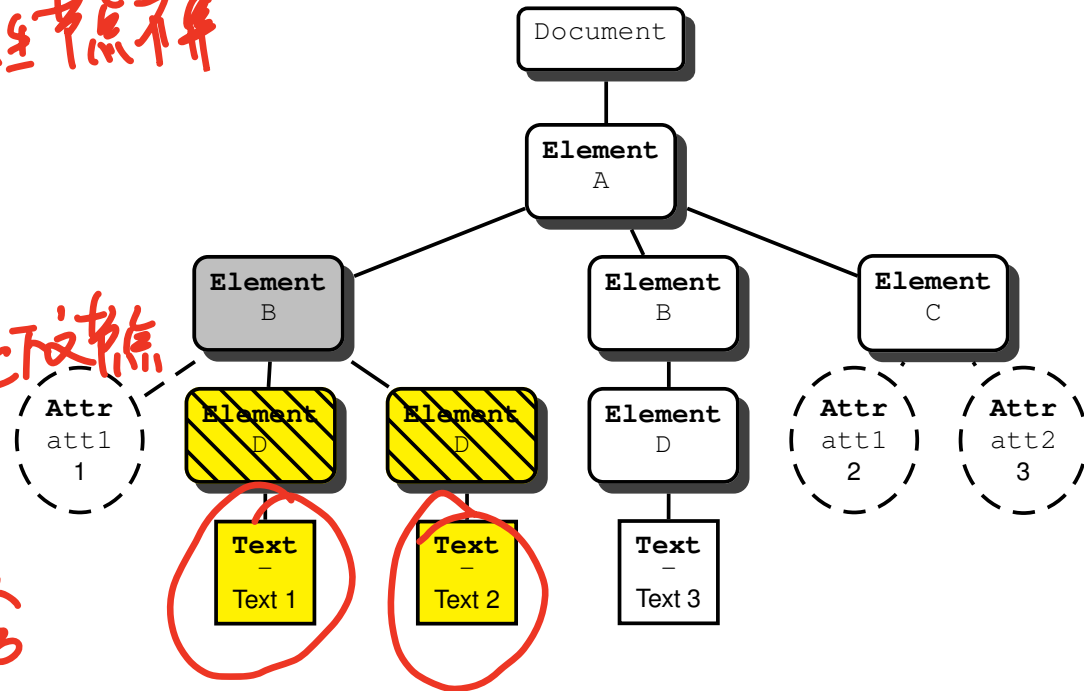
- All the descendant nodes, except the Attribute nodes
- Node test is either the node name (for **Element** nodes), or `*` (any **Element** node) or `text()` (any **Text** node) or `node()` (all nodes)
- The context node does not belong to the result: use descendant-or-self instead

属性节点不算

不包含上下文节点

包含

Result of `descendant::node()`

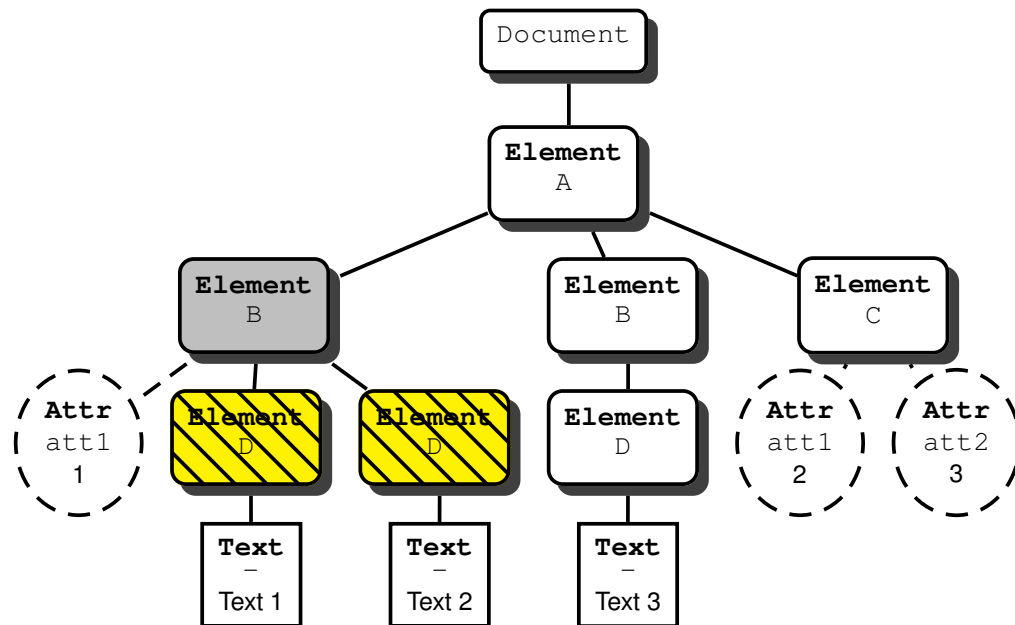


Examples of axis interpretation

Descendant axis:

- All the descendant nodes, **except** the **Attribute** nodes
- Node test is either the node name (for **Element** nodes), or ***** (any **Element** node) or **text()** (any **Text** node) or **node()** (all nodes)
- The context node does **not** belong to the result: use **descendant-or-self** instead

Result of **descendant** : ***** 元素

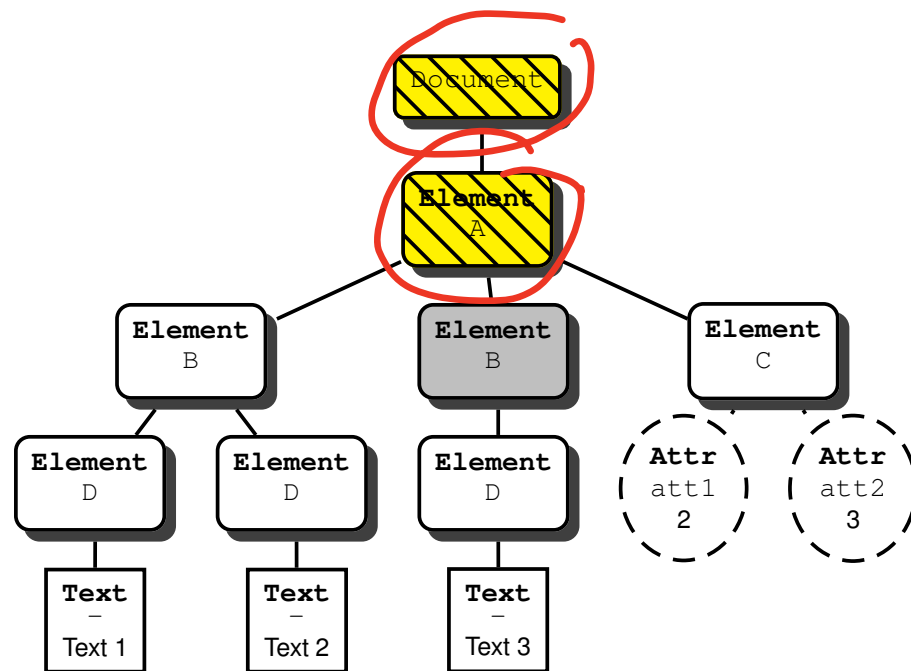


Examples of axis interpretation

Ancestor axis:

- All the ancestor nodes
- Node test is either the node name (for **Element** nodes), or `node()` (any **Element** node, and the **Document** root node)
- The context node does **not** belong to the result: use `ancestor-or-self` instead

Result of `ancestor::node()`

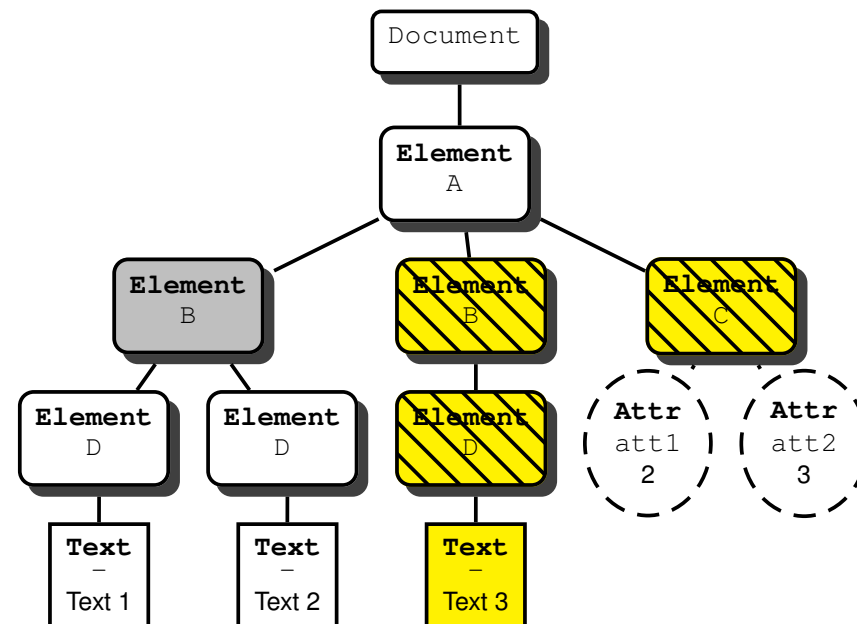


Examples of axis interpretation

Following axis:

- All the nodes that follow the context node in document order
- **Attribute** nodes are *not* selected
- The node test is either the node name, `*`, `text()` or `node()`
- The axis `preceding` denotes all the nodes that precede the context node

Result of `following::node()`



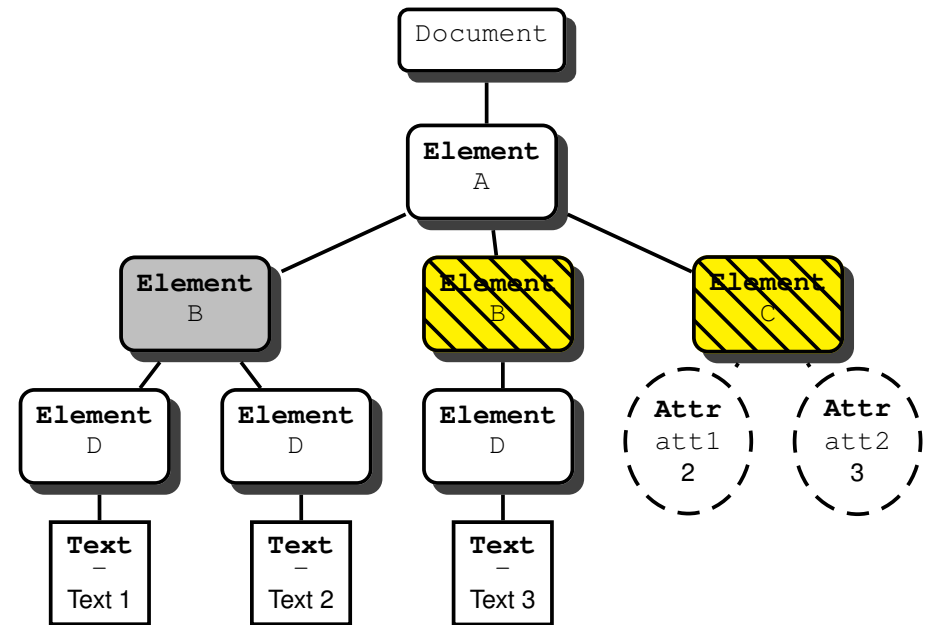
Examples of axis interpretation

Following sibling axis:

- All the nodes that follow the context node, and share the same parent node
- Same node tests as descendant and following
- The axis preceding-sibling denotes all the nodes that precede the context node

之后
之前

Result of `following-sibling::node()`



Abbreviations (summary)

Summary of abbreviations:

<code>somename</code>	<code>child::somename</code>	默认子访问
<code>.</code>	<u><code>self::node()</code></u>	自我缩写
<code>..</code>	<u><code>parent::node()</code></u>	
<u><code>@someattr</code></u>	<u><code>attribute::someattr</code></u>	
相对路径 ← <code>a//b</code>	<code>a/descendant-or-self::node()/b</code>	
<code>//a</code>	<code>/descendant-or-self::node()/a</code>	
<code>/</code>	<code>/self::node()</code>	

Abbreviations (summary)

Summary of abbreviations:

// 当前节点的所有后代, 包括自己

<code>somename</code>	<code>child::somename</code>	
<code>.</code>	<code>self::node()</code>	→ 上下文节点
<code>..</code>	<code>parent::node()</code>	父节点
<code>@someattr</code>	<code>attribute::someattr</code>	
绝对路径 <code>a//b</code>	<code>a/descendant-or-self::node()/b</code>	a元素后代中(包含a)所有的b元素
相对路径 <code>//a</code>	<code>/descendant-or-self::node()/a</code>	文档中所有b元素
<code>/</code>	<code>/self::node()</code>	根节点
		选中当前节点

Examples

- `@b` selects the `b` attribute of the context node
- `../*` selects all element siblings of the context node, itself included (if it is an element node)
- `//@someattr` selects all `someattr` attributes wherever their position in the document

Node Tests (summary)

A node test has one of the following forms:

- `node()` any node
- `text()` any text node
- `*` any element (or any attribute for the `attribute` axis)
- `ns:*` any element or attribute in the namespace bound to the prefix `ns` 前缀
- `ns:toto` any element or attribute in the namespace bound to the prefix `ns` and whose name is `toto`

Node Tests (summary)

A node test has one of the following forms:

- `node()` any node
- `text()` any text node
- `*` any element (or any attribute for the `attribute` axis)
- `ns:*` any element or attribute in the namespace bound to the prefix `ns`
- `ns:toto` any element or attribute in the namespace bound to the prefix `ns` and whose name is `toto`

Examples

- `a/node()` selects all nodes which are children of a `a` node, itself child of the context node
- `xsl:*` selects all elements whose namespace is bound to the prefix `xsl` and that are children of the context node
- `/` selects the top-level element node

所有直接子元素节点

从根节点向下一层全部

谓词

XPath Predicates

- Boolean expression, built with **tests** and the Boolean connectors `and` `and` `or` (negation is expressed with the `not ( )` function);
- A **test** is
 - ▶ Either an XPath expression, whose result is converted to Boolean
 - ▶ A comparison or call to a Boolean function

否定

XPath Predicates

- Boolean expression, built with **tests** and the Boolean connectors `and` and `or` (negation is expressed with the `not ( )` function);
- A **test** is
 - ▶ Either an XPath expression, whose result is converted to Boolean
 - ▶ A comparison or call to a Boolean function

Important: predicate evaluation applies several rules for converting nodes and node sets to the appropriate type

XPath Predicates

- Boolean expression, built with **tests** and the Boolean connectors `and` and `or` (negation is expressed with the `not()` function);
- A **test** is
 - ▶ Either an XPath expression, whose result is converted to Boolean
 - ▶ A comparison or call to a Boolean function

Important: predicate evaluation applies several rules for converting nodes and node sets to the appropriate type

Example

check true

- `//B[@att1=1]`: nodes `B` having an attribute `att1` with value 1;
- `//B[@att1]`: all nodes `B` having an attribute named `att1`!
 $\Rightarrow$ `@att1` is an XPath expression whose result (a node set) is converted to a Boolean
- `//B/descendant::text()[position()=1]`: the first **Text** node descendant of each node `B`; can be abbreviated by `//B/descendant::text()[1]`

缩写

Predicate evaluation

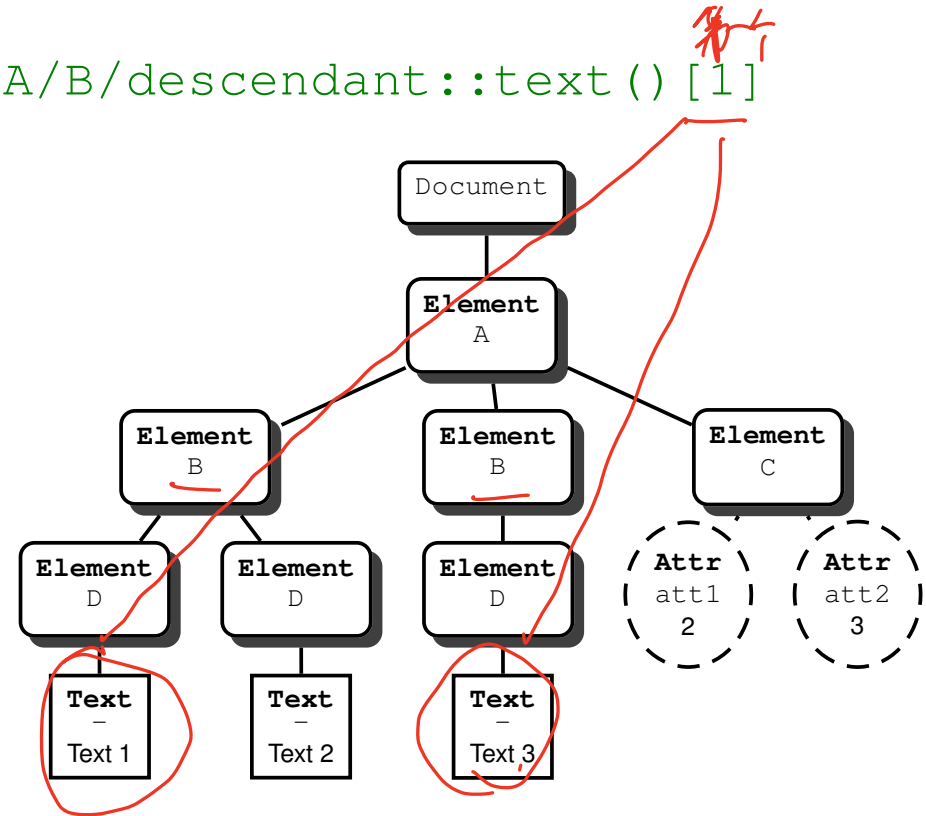
返回每一个B元素的第一个后代节点。

`/A/B/descendant::text()[1]` ^{第一个}

A step is of the form

`axis::node-test[P]`.

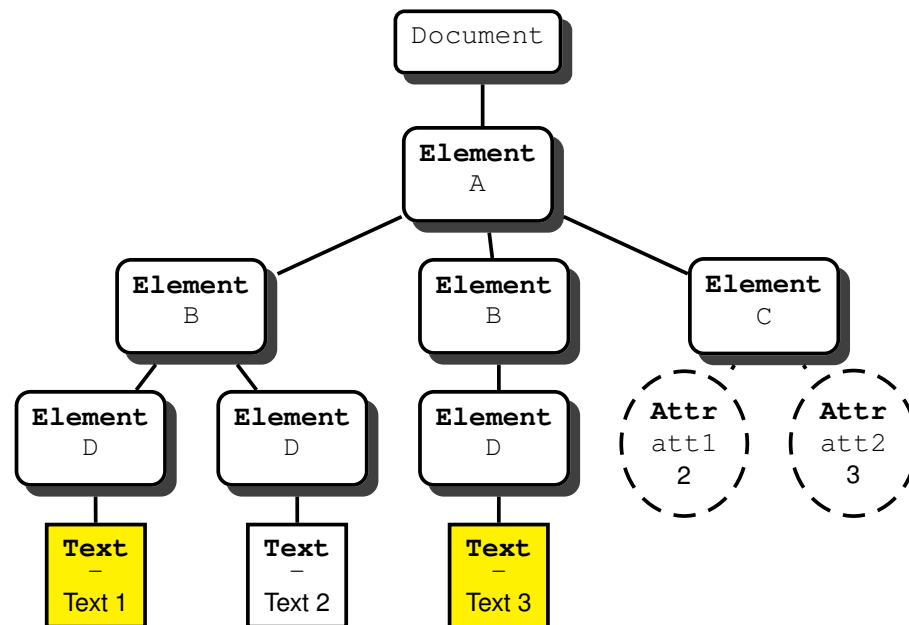
- First `axis::node-test` is evaluated: one obtains an intermediate result *I*
- Second, for each node in *I*, *P* is evaluated: the step result consists of those nodes in *I* for which *P* is true.



Predicate evaluation

Beware: An XPath step is **always** evaluated with respect to context of previous step
Here, the result consists of those **Text** nodes that are first descendants (in document order) of a node **B**

Result of `/A/B/descendant::text()[1]`



XPath 1.0 Type System

Four primitive types:

Type	Description	Literals	Examples
boolean	Boolean values	<i>none</i>	<code>true()</code> , <code>not(\$a=3)</code>
number	<u>Floating-point</u>	12, 12.5	<code>1 div 33</code>
string	Character strings	"to", 'ti'	<code>concat('Hello','!')</code>
nodeset	Node set	<i>none</i>	<code>/a/b[c=1 or @e]/d</code>

XPath 1.0 Type System

Four primitive types:

Type	Description	Literals	Examples <i>variables</i>
boolean	Boolean values	<i>none</i>	<code>true()</code> , <code>not(\$a=3)</code>
number	Floating-point	12, 12.5	<code>1 div 33</code>
string	Character strings	"to", 'ti'	<code>concat('Hello', '!')</code>
nodeset	Node set	<i>none</i>	<code>/a/b[c=1 or @e]/d</code>

The `boolean()`, `number()`, `string()` functions **convert** types into each other (no conversion to `nodeset` is defined), but this conversion is done in an **implicit** way most of the time

XPath 1.0 Type System

Four primitive types:

Type	Description	Literals	Examples
<code>boolean</code>	Boolean values	<i>none</i>	<code>true()</code> , <code>not(\$a=3)</code>
<code>number</code>	Floating-point	12, 12.5	<code>1 div 33</code>
<code>string</code>	Character strings	"to", 'ti'	<code>concat('Hello', '!')</code>
<code>nodeset</code>	Node set	<i>none</i>	<code>/a/b[c=1 or @e]/d</code>

The `boolean()`, `number()`, `string()` functions **convert** types into each other (no conversion to `nodeset` is defined), but this conversion is done in an **implicit** way most of the time

Rules for **converting to boolean**:

- A number is true if it is neither 0 nor NaN. 会被认为是 true, 除非是 0 或非数字
- A string is true if its length is not 0. 长度非0
- A nodeset is true if it is not empty.

Rules for converting nodeset to string:

节点 → 字符串

- The string value of a nodeset is the string value of its first item in document order
- The string value of an element or document node is the concatenation of the character data in all text nodes below
- The string value of a text node is its character data
- The string value of an attribute node is the attribute value

连接

所有文本
连接

属性值

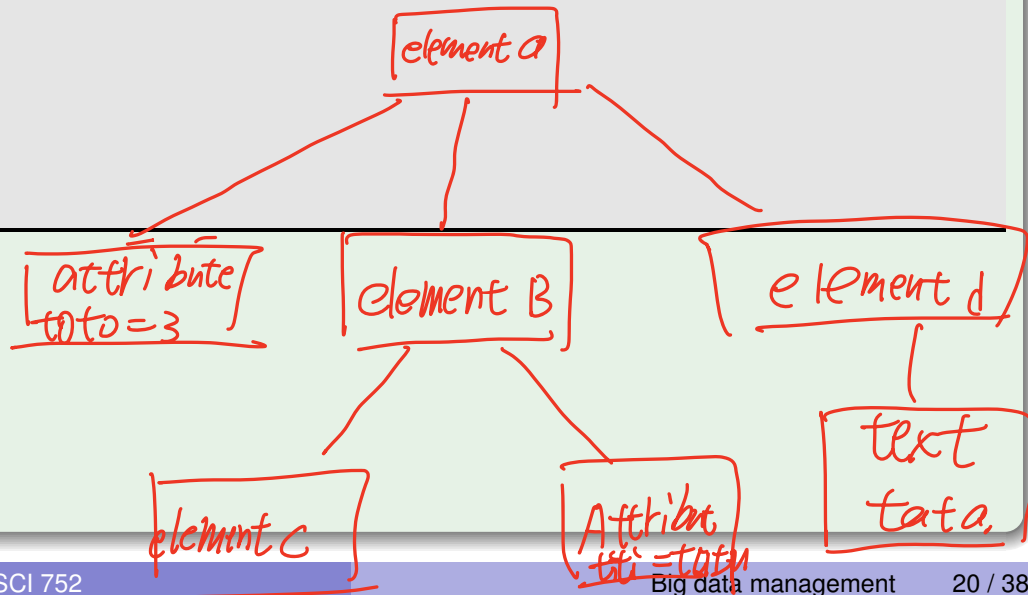
Rules for **converting nodeset to string**:

- The string value of a **nodeset** is the **string value** of its **first item** in document order
- The string value of an element or document node is the concatenation of the character data in **all text nodes** below
- The string value of a text node is its character data
- The string value of an attribute node is the attribute value

Examples (Whitespace-only text nodes removed)

```
<a toto="3">
  <b titi='tutu'><c /></b>
  <d>tata</d>
</a>
```

<code>string(/)</code>	"tata"
<code>string(/a/@toto)</code>	"3"
<code>boolean(/a/b)</code>	true()
<code>boolean(/a/e)</code>	false()



Outline

- 1 Introduction
- 2 Path Expressions
- 3 Operators and Functions**
- 4 XPath examples
- 5 XPath 2.0, 3.0, and 3.1
- 6 Reference Information
- 7 Exercise

Operators

The following operators can be used in XPath.

- `+`, `-`, `*`, `div`, `mod`: standard arithmetic operators (Example: `1+2*-3`); **Warning!** `div` is used instead of the usual `/`
ID = ID, node = node
- `or`, `and`: boolean operators (Example: `@a and c=3`)
- `=`, `!=`: equality operators. Can be used for strings, booleans or numbers; **Warning!** `//a!=3` means: there is an `a` element in the document whose string value is different from 3
- `<`, `<=`, `>=`, `>`: relational operators (Example: `($a<2) and ($a>0)`); **Warning!** Can only be used to compare numbers, not strings; If an XPath expression is embedded in an XML document, `<` must be escaped as `<`;
- `|`: union of nodesets (Example: `node() | @*`)
union

Operators

The following operators can be used in XPath.

- `+`, `-`, `*`, `div`, `mod`: standard arithmetic operators (Example: `1+2*-3`); **Warning!** `div` is used instead of the usual `/`
- `or`, `and`: boolean operators (Example: `@a and c=3`)
- `=`, `!=`: equality operators. Can be used for strings, booleans or numbers; **Warning!** `//a!=3` means: there is an `a` element in the document whose string value is different from 3
- `<`, `<=`, `>=`, `>`: relational operators (Example: `($a<2) and ($a>0)`); **Warning!** Can only be used to compare numbers, not strings; If an XPath expression is embedded in an XML document, `<` must be escaped as `<`;
- `|`: union of nodesets (Example: `node()|@*`)

Remark

`$a` is a **reference** to the variable `a`

Variables cannot be defined in XPath, they can only be referred to

Node Functions

- `count ($s)` : returns the **number of items** in the nodeset `$s`
- `local-name ($s)` : returns the **name** of the first item of the nodeset `$s` in document order, **without** the namespace prefix; if `$s` is omitted, it is taken to be the context item
- `namespace-uri ($s)` : returns the **namespace URI** bound to the prefix of the name of the first item of the nodeset `$s` in document order; if `$s` is omitted, it is taken to be the context item
- `name ($s)` : returns the **name** of the first item of the nodeset `$s` in document order, **including** its namespace prefix; if `$s` is omitted, it is taken to be the context item

String Functions

- `concat($s1, ..., $sn)`: **concatenates** the strings $s_1, \dots, s_n$
- `starts-with($a, $b)`: returns `true()` if the string a **starts with** b
- `contains($a, $b)`: returns `true()` if the string a **contains** b
- `substring-before($a, $b)`: returns the **substring** of a **before** first occurrence of b
- `substring-after($a, $b)`: returns the **substring** of a **after** the first occurrence of b
- `substring($a, $n, $l)`: returns the **substring** of a of length l starting at index n (indexes start from 1); l may be omitted
- `string-length($a)`: returns the **length** of the string a
- `normalize-space($a)`: **removes** all leading and trailing **whitespace** from a , and **collapse** all whitespace to a single character
- `translate($a, $b, $c)`: returns the string a , where all occurrences of a character from b have been **replaced** by the character at the same place in c

Boolean and Number Functions

- `not ($b)` : returns the **logical negation** of the boolean `$b`
- `sum ($s)` : returns the **sum** of the values of the nodes in the nodeset `$s`
- `floor ($n)` : rounds the number `$n` to the **next lowest** integer
- `ceiling ($n)` : rounds the number `$n` to the **next greatest** integer
- `round ($n)` : rounds the number `$n` to the **closest** integer

Boolean and Number Functions

- `not ($b)` : returns the **logical negation** of the boolean `$b`
- `sum ($s)` : returns the **sum** of the values of the nodes in the nodeset `$s`
- `floor ($n)` : rounds the number `$n` to the **next lowest** integer
- `ceiling ($n)` : rounds the number `$n` to the **next greatest** integer
- `round ($n)` : rounds the number `$n` to the **closest** integer

Examples

- `count (//*)` : returns the number of elements in the document
- `normalize-space(' titi toto ')` : returns the string "titi toto"
- `translate('baba','abcdef','ABCDEF')` : returns the string "BABA"
- `round(3.457)` : returns the number 3

Outline

- 1 Introduction
- 2 Path Expressions
- 3 Operators and Functions
- 4 XPath examples**
- 5 XPath 2.0, 3.0, and 3.1
- 6 Reference Information
- 7 Exercise

Examples (1)

- `child::A/descendant::B`:
B elements, descendant of an A element, itself child of the context node; Can be abbreviated to `A//B`
- `child::* / child::B`: all the B grand-children of the context node;
- `descendant-or-self::B`: elements B descendants of the context node, **plus** the context node itself if its name is B
- `child::B[position()=last()]`: the last child named B of the context node;
Abbreviated to `B[last()]`
- `following-sibling::B[1]`: the first sibling of type B (in the document order) of the context node;

Examples (2)

选第+个B. , 非第+个是不是B,

- `/descendant::B[10]`: the tenth element of type `B` in the document

Not: the tenth element of the document, if its type is `B`!

- ✓ • `child::B[child::C]`: child elements `B` that have a child element `C`;
Abbreviated to `B[C]`

- `/descendant::B[@att1 or @att2]`: elements `B` that have an attribute `att1` or an attribute `att2`;
Abbreviated to `//B[@att1 or @att2]`

- `(*)[self::B or self::C]:` children elements named `B` or `C`

Outline

- 1 Introduction
- 2 Path Expressions
- 3 Operators and Functions
- 4 XPath examples
- 5 XPath 2.0, 3.0, and 3.1**
- 6 Reference Information
- 7 Exercise

XPath 2.0 (W3C Recommendation since 23 January 2007)

An extension of XPath 1.0, backward compatible with XPath 1.0. Main differences:

- **Improved data model**: tightly associated with XML Schema.
 - ⇒ a new **sequence** type, representing ordered set of nodes and/or values, with duplicates allowed.
 - ⇒ XSD types can be used for node tests.
- **More powerful**: new operators (loops) and better control of the output (limited tree restructuring capabilities)
- **Extensible**: Many new built-in functions; possibility to add user-defined functions

XPath 2.0 (W3C Recommendation since 23 January 2007)

An extension of XPath 1.0, backward compatible with XPath 1.0. Main differences:

- **Improved data model**: tightly associated with XML Schema.
 - ⇒ a new **sequence** type, representing ordered set of nodes and/or values, with duplicates allowed.
 - ⇒ XSD types can be used for node tests.
- **More powerful**: new operators (loops) and better control of the output (limited tree restructuring capabilities)
- **Extensible**: Many new built-in functions; possibility to add user-defined functions

XPath 2.0 is **also** a **subset** of XQuery 1.0.

Path expressions in XPath 2.0

New node tests in XPath 2.0:

- `item()`: any node or atomic value
- `element()`: any element (eq. to `child::*` in XPath 1.0)
- `element(author)`: any element named `author`
- `element(*, xs:person)`: any element of type `xs:person`
- `attribute()`: any attribute

Path expressions in XPath 2.0

New node tests in XPath 2.0:

- `item()`: any node or atomic value
- `element()`: any element (eq. to `child::*` in XPath 1.0)
- `element(author)`: any element named `author`
- `element(*, xs:person)`: any element of type `xs:person`
- `attribute()`: any attribute

Nested paths expressions:

Any expression that returns a sequence of nodes can be used as a step.

```
/book/(author | editor)/name
```

Loops in XPath 2.0

迭代

- Provide the ability to iterate over sequences
- Like XSLT `<xsl:for-each>` except inside an XPath expression
- Apply an expression to every item in a sequence

Loops in XPath 2.0

- Provide the ability to iterate over sequences
- Like XSLT `<xsl:for-each>` except inside an XPath expression
- Apply an expression to every item in a sequence

```
for $n in child::name return concat($n/fname,' ', $n/surname)
```

XPath 3.0 & 3.1 (W3C recommendation since 8 April 2014 & 21 March 2017)

- Inline function expressions, e.g. `function($a as xs:double,$b as xs:double) as xs:double{$a*$b}`
- Dynamic function calls, e.g. `$f[2]("Hi there")` fetches the second item from sequence `$f`, treats it as a function and invokes it, passing an `xs:string` argument
- Union types, as defined in XML Schema
- Namespace literals that reference XML namespaces by using braced URI literals
- String concatenation operator, e.g. `$a||$b` is equivalent to `fn:concat($a,$b)`
- Mapping operator, e.g. `E1!E2` evaluates `E2` for each item in the sequence `E1`, and concatenates the resulting items (no duplicate elimination nor document ordering)
- More type support in XPath 3.1 for arrays (key-value pairs) used by JSON
- XPath 3.1 arrow operator `=>` for function chaining, where

`contains(upper-case(substring-before($in,' ')), 'X')`

becomes

`$in=>substring-before(' ')=>upper-case()=>contains('X')`

Outline

- 1 Introduction
- 2 Path Expressions
- 3 Operators and Functions
- 4 XPath examples
- 5 XPath 2.0, 3.0, and 3.1
- 6 Reference Information**
- 7 Exercise

XPath 1.0 Implementations

Large number of implementations.

- [libxml2](#): Free **C** library for parsing XML documents, supporting XPath
- [java.xml.xpath](#): **Java** package, included with JDK versions starting from 1.5
- [System.Xml.XPath](#): **.NET** classes for XPath
- [XML::XPath](#): Free **Perl** module, includes a command-line tool
- [DOMXPath](#): **PHP** class for XPath, included in PHP5
- [PyXML](#): Free **Python** library for parsing XML documents, supporting XPath

References

- `http://www.w3.org/TR/xpath`
- *XML in a nutshell*, Eliotte Rusty Harold & W. Scott Means, O'Reilly
- *XPath 2.0 Programmer's Reference*, Michael Kay, Wrox

Outline

- 1 Introduction
- 2 Path Expressions
- 3 Operators and Functions
- 4 XPath examples
- 5 XPath 2.0, 3.0, and 3.1
- 6 Reference Information
- 7 Exercise**

Exercise

```
<a>
  <b><c /></b>
  <b id="3" di="7">bli <c /><c><e>bla</e></c></b>
  <d>bou</d>
</a>
```

We suppose that all text nodes containing only whitespace are removed from the tree.

- Give the result of the following XPath expressions:

- ▶ `//e/preceding::text()`:

bli

Exercise

```
<a>
  <b><c /></b>
  <b id="3" di="7">bli <c /><c><e>bla</e></c></b>
  <d>bou</d>
</a>
```

We suppose that all text nodes containing only whitespace are removed from the tree.

- Give the result of the following XPath expressions:

- ▶ `//e/preceding::text():"bli"`
- ▶ `count(//c|//b/node()):`

Exercise

```
<a>
  <b><c /></b>
  <b id="3" di="7">bli <c /><c><e>bla</e></c></b>
  <d>bou</d>
</a>
```

We suppose that all text nodes containing only whitespace are removed from the tree.

- Give the result of the following XPath expressions:

- ▶ `//e/preceding::text():"bli"`

- ▶ `count(//c|//b/node()):4`

去重后结果:

- Give an XPath expression for the following problems, and the corresponding result:

- ▶ Sum of all attribute values:

Exercise

```
<a>
  <b><c /></b>
  <b id="3" di="7">bli <c /><c><e>bla</e></c></b>
  <d>bou</d>
</a>
```

We suppose that all text nodes containing only whitespace are removed from the tree.

- Give the result of the following XPath expressions:
 - ▶ `//e/preceding::text():"bli"`
 - ▶ `count(//c|//b/node()):4`
- Give an XPath expression for the following problems, and the corresponding result:
 - ▶ Sum of all attribute values: `sum(//@*)=10`
 - ▶ Text content of the document, where every "b" is replaced by a "c":

Exercise

```
<a>
  <b><c /></b>
  <b id="3" di="7">bli <c /><c><e>bla</e></c></b>
  <d>bou</d>
</a>
```

We suppose that all text nodes containing only whitespace are removed from the tree.

- Give the result of the following XPath expressions:
 - ▶ `//e/preceding::text():"bli "`
 - ▶ `count(//c|//b/node()):4`
- Give an XPath expression for the following problems, and the corresponding result:
 - ▶ Sum of all attribute values: `sum(//@*)=10`
 - ▶ Text content of the document, where every "b" is replaced by a "c":
`translate(string(/),'b','c')="cli clacou"`
 - ▶ Name of the child of the last "c" element in the tree:

Exercise

```
<a>
  <b><c /></b>
  <b id="3" di="7">bli <c /><c><e>bla</e></c></b>
  <d>bou</d>
</a>
```

We suppose that all text nodes containing only whitespace are removed from the tree.

- Give the result of the following XPath expressions:
 - ▶ `//e/preceding::text():"bli "`
 - ▶ `count(//c|//b/node()):4`
- Give an XPath expression for the following problems, and the corresponding result:
 - ▶ Sum of all attribute values: `sum(//@*)=10`
 - ▶ Text content of the document, where every “b” is replaced by a “c”:
`translate(string(/),'b','c')="cli clacou"`
 - ▶ Name of the child of the last “c” element in the tree: `name(//c[last()]/*[1])="e"`